Course: CS 4850 Senior Project
Semester: Fall 2025
Project Title: INDY 500 RiverGuard

Team Members:

Collin Tucker - Team Lead

Pedro Pinto - Information Architect and Software Engineer

Geshlee Ruiz - Software Engineer

Grant Versluis - Software Architect

Wyatt Bramblett - Agile Project Coordinator

Advisor: Sharon Perry

GitHub Link: https://github.com/CollinT123/RiverGuard

NDA Status: This project is not operating under an NDA

Website: https://river-guard.vercel.app/project

Metric	Value
Lines of Code	5,000
Components Used	12
Hours Estimated	250
Hours Actual	300
Completion Percent	100%

Table of Contents

1 Introduction

1.1 Overview

RiverGuard is a computer vision system designed to detect trash in river footage. The project uses uploaded videos, extracts frames, and analyzes those frames with a YOLOv12 model trained on water trash datasets. The detection results are stored in Firestore and displayed through a Next.js web dashboard. The purpose of the system is to help environmental organizations collect consistent data about pollution levels without relying on manual observation.

1.2 Objectives

The main objective of RiverGuard is to demonstrate that automated trash detection can support pollution monitoring and trend analysis. A secondary objective is creating a modular architecture where each layer of the system can be scaled or replaced. The final objective is producing a functioning MVP that can be demoed to instructors or stakeholders.

1.3 Technologies

The system uses several major technologies that work together to form the full pipeline. These include Next.js for the UI, Express for routing, Docker for containerized processing, FFmpeg for frame extraction, a YOLOv12 model for detection, and Firebase for storage.

Technology / Framework	Purpose
Next.js	Provides the frontend interface for uploads and dashboard visualization
React Components	Manages UI rendering and state handling
Express.js	Handles middleware routing and API communication
Nginx	Directs traffic as a reverse proxy for frontend and backend services
Docker	Containerizes backend subsystems for stability and isolation
FFmpeg	Extracts frames from uploaded video files
YOLOv12	Performs object detection for identifying trash in frames
Python	Implements model server logic and inference handling
Firebase	Manages initialization configuration and application services
Firestore	Stores detection metadata including timestamps and counts
Firebase Storage	Stores annotated frames when privacy settings permit
Linux Server	Hosts backend containers and provides the deployment environment

2 Requirements

2.1 Functional Requirements

RiverGuard must support the complete flow from video upload to dashboard display. The system must validate uploaded files, extract frames, run object detection, store results, and display them. Functional requirements include:

1. Video upload and format validation
2. Frame extraction using FFmpeg
3. YOLOv12 detection on all extracted frames
4. Storage of timestamps, counts, and model metadata
5. Dashboard that displays detection statistics and optional annotated frames
6. Clear error messages for invalid uploads or processing failures

2.2 Non-Functional Requirements

The system must remain secure, understandable, and reliable. Nonfunctional requirements include:

1. Restricted access to management-level features
2. A user interface suitable for non-technical users
3. Support for parallel processing depending on hardware
4. Acceptable latency between upload and result
5. Automatic recovery from container failures

Requirement Type	Description
Functional	Validate uploaded video file formats
Functional	Extract video frames using FFmpeg
Functional	Perform YOLOv12 detection on all extracted frames
Functional	Store detection metadata in Firestore
Functional	Display detection statistics and annotations on dashboard
Functional	Provide error messages for invalid uploads or processing failures
Non-Functional	Restrict system level access to authorized users
Non-Functional	Maintain a UI accessible to non-technical users
Non-Functional	Support multiple video processing tasks
Non-Functional	Maintain reasonable processing latency
Non-Functional	Automatically recover from backend container failures

3 Analysis and Design

3.1 Architectural Overview

RiverGuard is organized into a frontend, middleware, backend, model server, and Firestore database. The frontend collects uploads and displays analytics. The middleware receives upload requests and forwards them to the backend. The backend uses Docker containers to isolate processing tools. One container extracts frames using FFmpeg. Another container formats images. The YOLOv12 model container performs detection and writes results directly to Firestore. This modular design improves reliability and makes the system easier to maintain.

3.2 Design Considerations

Several constraints influenced the design. Firestore has limits on write frequency, so detection results must remain lightweight. FFmpeg operations can be CPU intensive, so videos must be handled efficiently. The YOLO model loads only once per container to avoid unnecessary overhead. Assumptions include the expectation that videos depict clear river scenes with visible trash objects.

3.3 Subsystem Summaries

The frontend uses React components to guide users through the upload flow and display detection summaries. The middleware acts as a communication layer between the user and the backend. The backend performs the main processing work, including video decoding and inference. The database stores structured detection records, including timestamps, counts, model version, and latency. These values enable users to understand how much trash was present at different points in the video.

Subsystem	Responsibilities
Frontend (Next.js)	Uploads river footage, displays system information, renders detection summaries and statistics
Middleware (API Routes + Express)	Receives uploaded videos, forwards data to backend, provides workflow updates
Backend (Docker Containers)	Extracts frames, prepares them for inference, handles communication between containers
Model Server (YOLOv12 Container)	Loads YOLO model, performs object detection, writes results to Firestore
Firestore Database	Stores timestamps, detection counts, model version, and optional frame data

4 Development

4.1 Components

RiverGuard was developed using Next.js for the UI, Express for middleware, and several Docker containers for processing. FFmpeg handles frame extraction. YOLOv12 performs object detection. Firestore stores the metadata. Firebase Storage optionally holds frame images if privacy settings allow.

4.2 Development Process

Development began with the frontend upload interface and dashboard. The backend was then created using Docker to isolate processing tasks. FFmpeg was integrated for reliable frame extraction. The YOLOv12 model container was built using Python so that frames could be analyzed efficiently. Middleware routing was added to ensure clean communication between components. Firestore integration completed the pipeline by allowing results to persist.

4.3 Database Interaction

The backend initializes a Firestore instance and writes detection output to the appropriate document paths. Each detection includes trash counts, frame timestamps, and model information. The dashboard reads this data in real time. Frame storage is optional and depends on privacy settings.

4.4 Setup Instructions

Steps to run RiverGuard:

1. Clone the repository
2. Install dependencies using npm install
3. Populate environment variables
4. Start middleware using npm run dev
5. Start backend containers with docker compose up build
6. Start the Next.js frontend using npm run start
7. Upload a test video and verify Firestore receives data

4.5 Version Control

The project uses GitHub for source management. The repository is organized into folders for frontend, middleware, backend infrastructure, and model scripts. Final source code will be delivered in a zip file.

4.6 Notable Challenges

Challenges included maintaining acceptable performance with FFmpeg, managing Firestore write limitations, and improving the model's accuracy under difficult lighting conditions. Upload cancellation scenarios also required extra handling to avoid creating partial or corrupted jobs.

Risk or Challenge	Description
FFmpeg Performance	Large videos may slow down frame extraction
Firestore Write Limits	High detection frequency increases write load
Model Accuracy Variations	Lighting and water clarity affect detection quality
Upload Reliability	Interrupted uploads may cause incomplete jobs
Container Failures	Model or processing containers may require automatic restart

5 Test Plan and Results

5.1 Test Plan

Tests followed the Software Test Plan document. The focus was confirming the reliability of the upload workflow, the accuracy of trash detection, and the stability of data storage.

5.2 Progression Tests

Progression testing was conducted to validate every major component of the YOLO-based pipeline. Test cases included uploading both valid and invalid video formats to ensure robust error handling, verifying that the backend correctly extracts frames at the configured sampling rate, and confirming that YOLO inference produces accurate bounding boxes and class predictions across a variety of lighting and motion conditions. Additional tests evaluated dashboard correctness by checking that detection counts, confidence levels, timestamps, and annotated previews were all displayed consistently with backend results. Multi-video batch processing was also validated to ensure the system could simultaneously process multiple uploads without performance degradation or race conditions. All progression tests passed successfully, confirming end-to-end functionality and system reliability.

5.3 Regression Tests

Regression tests ensured that changes to backend and frontend components did not reintroduce previously fixed issues. Regression items R1 through R5 passed successfully.

5.4 Test Case Results

Twelve test cases were executed and evaluated. These included upload behavior, invalid file handling, frame extraction consistency, model accuracy on both trash filled and clean videos, backend storage, dashboard loading, failure handling during container restarts, and stress conditions with multiple videos.

5.5 Defects

Two minor defects were identified, involving dashboard load time under high data volume and an upload cancellation message that disappeared too quickly. These do not affect core functionality.

5.6 Overall Evaluation

The system meets all acceptance criteria for the MVP stage and is ready for demonstration.

Test Case	Description	Result
TC 01	Upload a valid video file	Passed
TC 02	Reject unsupported or invalid file formats	Passed
TC 03	Extract frames using FFmpeg	Passed
TC 04	Perform YOLO inference on sample footage	Passed
TC 05	Detect zero trash in clean river footage	Passed
TC 06	Write and retrieve detection data from Firestore	Passed
TC 07	Display accurate dashboard data	Passed
TC 08	Process multiple videos sequentially	Passed
TC 09	Handle upload cancellation gracefully	Passed
TC 10	Recover from model container restart	Passed
TC 11	Load dashboard under higher data volume	Passed with minor delay
TC 12	Validate end to end pipeline behavior	Passed

6 Conclusion

The RiverGuard system successfully completes the full pipeline from video upload to trash detection and visualization. Each subsystem performs its role reliably, and the model provides consistent detection results in various conditions. The system offers a strong foundation for future improvements, such as real time livestream support, expanded trash categories, larger datasets for improved accuracy, and more sophisticated analytics features. RiverGuard achieves the intended goals and is suitable for presentation and evaluation.

7 Appendix

7.1 Training Verification

- Dataset Integrity Checks
    - Verified that all training, validation, and test images were correctly labeled and followed the expected folder structure.
    - Confirmed the absence of corrupted images, missing annotation files, and mismatched label dimensions.
    - Ensured class distribution balance and validated that all classes appearing in the dataset were registered in the YAML configuration.
- Training Stability Monitoring
    - Monitored loss curves (box loss, objectness loss, classification loss) to ensure they exhibited smooth convergence without divergence spikes.
    - Verified that the validation mAP improved across epochs and corresponded to improvements seen in training metrics.
    - Identified and checked for signs of overfitting using the gap between training and validation performance.
- Model Output Evaluation
    - Inspected predictions on a withheld validation set to ensure accurate bounding boxes, proper confidence scores, and correct class labels.
    - Verified annotation alignment by comparing predicted masks/boxes with known ground truth samples.
    - Confirmed correct handling of edge cases such as small objects, overlapping objects, and scenes with cluttered backgrounds.
- Reproducibility Checks
    - Re-ran model training with the same random seed to confirm reproducible results in terms of loss curves, mAP values, and inference quality
- Performance Benchmarks
    - Recorded inference speed (FPS), average detection time per frame, and GPU utilization during validation.
    - Verified that trained weights met or exceeded baseline performance metrics set at the beginning of the project.

7.2 Project Plan

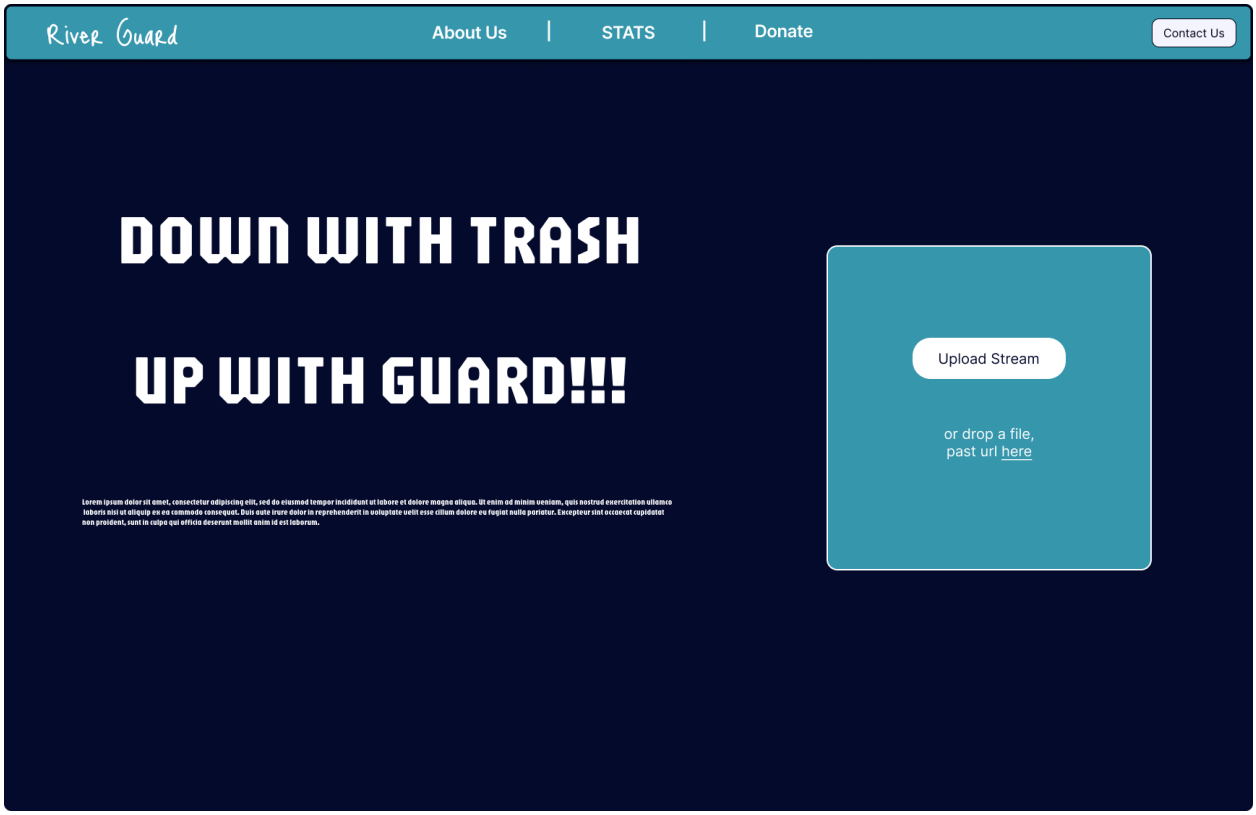
The RiverGuard project followed a structured development approach divided into frontend, backend, model training, and automation responsibilities. The team adopted a phased plan beginning with requirement analysis and initial architecture selection, followed by parallel development tracks. The frontend team built a Next.js interface allowing users to upload river videos or provide livestream URLs. The middleware utilized FFmpeg to extract

frames, which were then processed by Yolo V12 nano running inside isolated Docker containers to enable concurrent job execution. Firebase served as the backend for storing detection results and metadata.

In early planning discussions, the team agreed to begin with the model training, followed by a proof of concept for frame extraction. Automation planning included deploying containers to a Linux server and defining scripts to coordinate the full processing pipeline. Roles were clearly defined: Pedro led model training, Wyatt managed automation and containers, Collin handled the pipeline and helped with containerization, Geshlee managed the front end, and Grant integrated the middleware and backend onto a Nginx and Linux server respectively and helped with the front end. This project plan guided the execution of all deliverables throughout the semester.

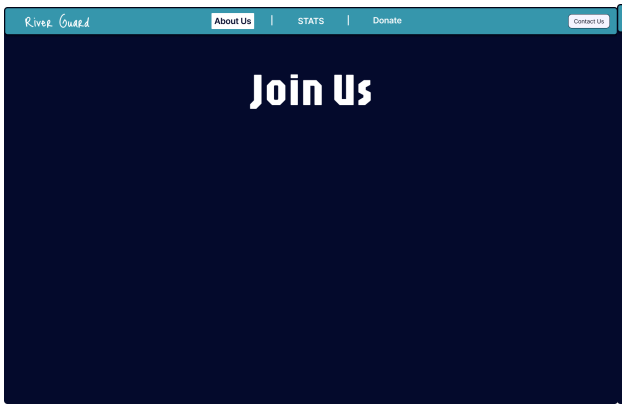
7.3 Screen Mockups

Home Page

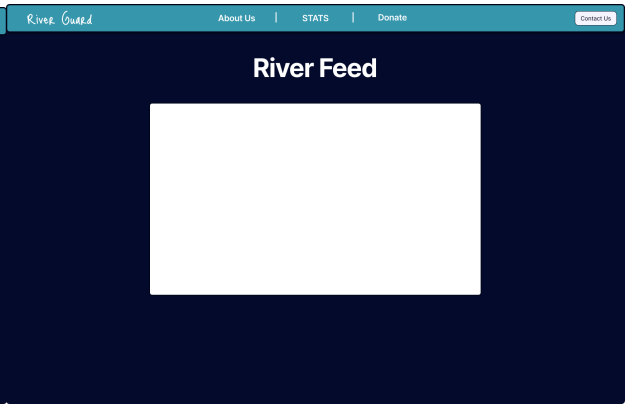


About Page

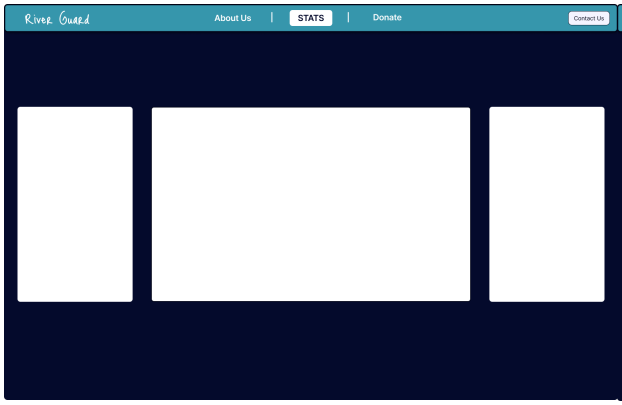
Video Feed Page

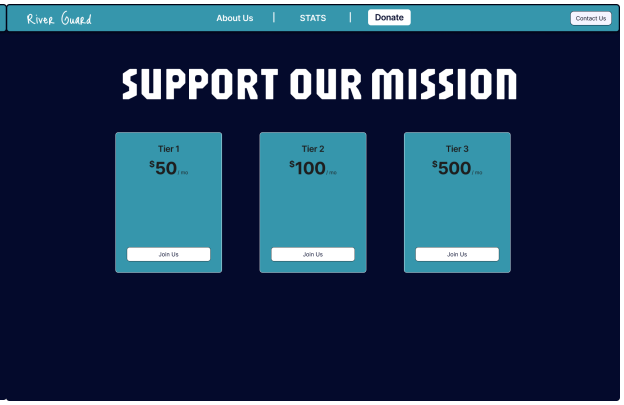


Statistics Page

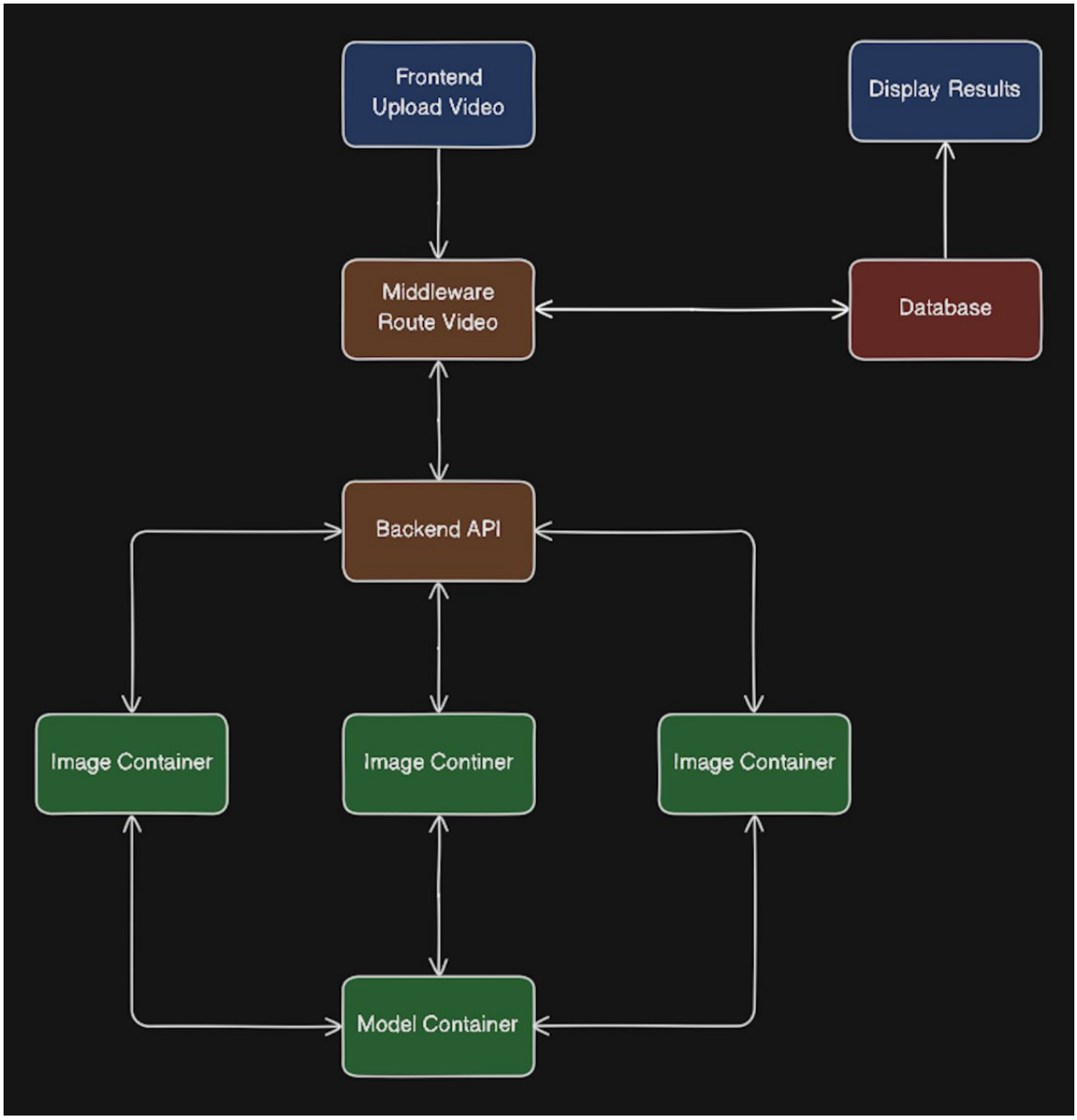


Donation Page





7.4 Architecture Diagrams



7.5 Supporting Documents

SRS - SRS_SeniorProject.docx
SDD - SDD_SeniorProject.docx
STP and STR - Indy500-RiverGuard-STP-STR
Development Document - DevelopmentDoc.docx